Acceptance Logging (Implementation Details)

Goal. Record sweep-level acceptance ratios without storing full trajectories in RAM.

1. **Per-sweep ratio.** In each sweep, the code attempts N single-particle moves and counts accepted moves a. The recorded value is a/N (a float per sweep).

2. **Recording windows.** When `n_samples = 0`, acceptance is recorded for *all* sweeps. When `n_samples > 0`, acceptance is recorded *only during sampling*: the first (unsaved) sampling window of length Δ, plus each saved window thereafter. Burn-in sweeps are not recorded.

3. **Storage.** Acceptance ratios are appended into a preallocated memmap array `acceptance.npy` with length

$$\texttt{acc_len} = \begin{cases} \texttt{sweeps} & \text{if } n_\text{samples} = 0 \\ \Delta \cdot (1 + n_\text{saved}) & \text{if } n_\text{samples} > 0 \end{cases}$$

   and sweep indices are stored in parallel in `acceptance_sweep.npy` as a simple 0-based counter. These indices refer to *recorded* sweeps (sampling-only), not absolute sweeps including burn-in.

4. **Summary metrics.** The run reports `acceptance_mean` as the average of recorded values, and stores `acceptance_count` (written length) and `acceptance_expected` (preallocated length) in the manifest metadata.

Random Seed Handling in `Ka2d_random_seed.py`

Goal. A single user seed controls all randomness: NumPy PCG64 for type shuffling and initialization, and Numba's legacy RNG for in-kernel `np.random.*` calls.

1. **Seed routing.** If `cfg.seed` is not `None`, build a `SeedSequence` and spawn three child streams: (*i*) `rng_types` for shuffling types, (*ii*) `rng_pos` for initial positions, (*iii*) `seed_numba_int` (a 32-bit integer) for Numba's legacy RNG. If `cfg.seed` is `None`, all RNGs are left unseeded (OS entropy).

2. **Python-side draws.** Types are shuffled with `rng_types.shuffle`; initial wrapped positions are drawn via `rng_pos.random((N,2)) * box`. These are reproducible for a fixed `cfg.seed`.

3. **Numba seeding.** A tiny `@njit` helper `_seed_numba(seed_int)` calls `np.random.seed(seed_int)`. It is invoked exactly once at the start of `run()` when `cfg.seed` is set, before any MC sweeps, and never reseeded afterward.

4. **Autotune policy.** Burn-in respects `cfg.autotune`; sampling forces `autotune=False` (kernel frozen) without mutating `cfg.autotune` so metadata remains truthful.

5. **Determinism scope.** Same `cfg.seed` yields identical types, initial positions, and MC trajectories/energies on CPU single-thread execution. Different seeds diverge; `seed=None` is nondeterministic. Parallel execution may reorder RNG calls and is not guaranteed deterministic.

Sample Storage and Appending

Goal. Save samples incrementally (append) so runs do not accumulate all samples in RAM and can resume or inspect partial output.

1. **Chunk directory layout.** Given a base path `<base>.npz`, the writer builds `<base>__chunks/` and reserves deterministic file names: `positions.npy`, `positions_real.npy`, optional `acceptance*.npy`, and single-instance metadata arrays (`types.npy`, `box.npy`, `initial_positions.npy`, `final_positions.npy` etc.).

2. **Preallocation via memmap.** `ChunkedStorage` opens each array with `np.lib.format.open_memmap(mode=` and an explicit shape $(n_\text{saved}, N, 2)$ (or acc_len). This allocates the on-disk array once and enables constant-time slice writes.

3. **Append semantics.** Each sample is written by `write_sample(pos, pos_real)`, which assigns into the preallocated memmap at index `sample_written` and increments the counter. No buffering of previous samples occurs, so memory stays bounded and order reflects simulation time.

4. **Acceptance trace.** If acceptance is requested, `write_acceptance` writes a block of values and records their sweep indices into `acceptance_sweep.npy`, again via slicing into the memmap.

5. **Manifest + pointer.** `finalize_manifest` flushes memmaps, writes a manifest JSON with file paths and metadata, and drops a tiny pointer JSON at the original base path for compatibility. Loading uses `load_full_output`, which reads the manifest and memory-maps the chunked arrays.

6. **Why this guarantees append.** Because arrays are preallocated to the target length and every sample write is an in-place slice assignment into a memmap, the code never keeps or rewrites the full sample tensor in memory. The manifest is the only aggregation step, executed after all slice writes complete.

Parallel KA2D with Chunked Logging (`Ka2d_random_seed_optimized_logging_a`

Parallel Batch Runner (`Ka2d_parallel_runner.py`)

Single-Run KA2D with Unified Seed (`Ka2d_random_seed.py`)

Algorithm 1 KA2D Metropolis Monte Carlo (burn-in, interval sampling; unwrapped trajectory)

Notation. $i \in \{1, \ldots, N\}$, types $\boldsymbol{\tau} = (\tau_i)$. Unwrapped positions $\mathbf{y}_i \in \mathbb{R}^2$ and wrapped positions $\mathbf{x}_i = \mathcal{W}_L(\mathbf{y}_i) \in [0, L)^2$, with componentwise wrap $\mathcal{W}_L(\mathbf{z}) = \mathbf{z} \bmod L$. Total energy $U(\mathbf{x}, \boldsymbol{\tau}; L)$ and single-particle energy change ΔU_i.

Require: N, ρ, T; potential params $\theta = (\varepsilon, \sigma, C_0, C_2, C_4, r_{\text{cut}})$; initial step size d_0, RNG seed; tuning $(\mathit{autotune}, \alpha^\star, \Delta_{\text{tune}})$; burn-in n_b, sample interval Δ, #samples S

Ensure: $\{(\mathbf{x}^{(s)}, \mathbf{y}^{(s)}, E^{(s)})\}_{s=0}^{S-1}$, $\boldsymbol{\tau}$, L, acceptance trace (optional)

$\beta \leftarrow 1/T$; $L \leftarrow \sqrt{N/\rho}$
$\boldsymbol{\tau} \leftarrow$ composition vector; random permutation (seeded)
$\mathbf{y}_i \sim \text{Unif}([0, L)^2)$; $\mathbf{x}_i \leftarrow \mathbf{y}_i$ ▷ $\mathbf{x}_i = \mathcal{W}_L(\mathbf{y}_i)$ initially
$E \leftarrow U(\mathbf{x}, \boldsymbol{\tau}; L)$; $d \leftarrow d_0$
$(E, d) \leftarrow \textsc{RunBlock}(n_b,\ E,\ d)$ ▷ burn-in
for $s = 0$ to $S - 1$ **do**
    $(E, d) \leftarrow \textsc{RunBlock}(\Delta,\ E,\ d)$
    $\mathbf{x}^{(s)} \leftarrow \mathbf{x}$; $\mathbf{y}^{(s)} \leftarrow \mathbf{y}$; $E^{(s)} \leftarrow E$
end for

Subroutines.

function $\textsc{RunBlock}(n,\ E,\ d)$
    for $t = 1$ to n **do**
        $(E, a) \leftarrow \textsc{Sweep}(E,\ d)$
        record a/N (optional; see Acceptance Logging)
        if $\mathit{autotune} \wedge (t \bmod \Delta_{\text{tune}} = 0)$ **then**
            $\bar{\alpha} \leftarrow$ mean acceptance over last Δ_{tune} sweeps
            $d \leftarrow d \cdot \big(0.9\,\mathbb{I}[\bar{\alpha} < \alpha^\star] + 1.1\,\mathbb{I}[\bar{\alpha} \geq \alpha^\star]\big)$
            $d \leftarrow \min(\max(d, 0.01), 0.8)$
        end if
    end for
    return (E, d)
end function
function $\textsc{Sweep}(E,\ d)$
    $a \leftarrow 0$
    for $m = 1$ to N **do** ▷ N trial moves / sweep
        $i \sim \text{Unif}\{1, \ldots, N\}$
        $\boldsymbol{\delta} \sim \text{Unif}([-d, d]^2)$
        $\mathbf{y}_i^\star \leftarrow \mathbf{y}_i + \boldsymbol{\delta}$; $\mathbf{x}_i^\star \leftarrow \mathcal{W}_L(\mathbf{x}_i + \boldsymbol{\delta})$
        $\Delta U_i \leftarrow U_i(\mathbf{x}_i^\star; \mathbf{x}_{\neg i}, \boldsymbol{\tau}, L) - U_i(\mathbf{x}_i; \mathbf{x}_{\neg i}, \boldsymbol{\tau}, L)$
        $\alpha \leftarrow \min\Big(1, e^{-\beta\,\Delta U_i}\Big)$; $u \sim \text{Unif}[0, 1]$
        if $u < \alpha$ **then**
            $\mathbf{x}_i \leftarrow \mathbf{x}_i^\star$; $\mathbf{y}_i \leftarrow \mathbf{y}_i^\star$; $E \leftarrow E + \Delta U_i$; $a \leftarrow a + 1$
        end if
    end for
    return (E, a)
end function

Algorithm 2 Single run with progress logging and chunked sample writes

Build params θ and config cfg; derive counts, box L, shuffled types, initial positions; precompute r^2_{cut}, energy E
Allocate `ChunkedStorage` (memmap arrays sized to $n_$samples and acceptance length) and `ProgressLogger`
Seed Numba RNG once (if $cfg.seed$ provided); set $d \leftarrow cfg.max_disp$, $\beta \leftarrow 1/T$
Burn-in: Run $cfg.burn_in_sweeps$ with autotune; skip recording to save memory; logger records window summaries
Sampling: For each window of $cfg.sample_interval$ sweeps:
  freeze autotune; run interval; append wrapped and real positions via `write_sample`; optionally append acceptance window
  log sweep, samples written, acceptance mean
Flush memmaps; write manifest JSON + pointer; close logger; return metadata (final d, acceptance stats, manifest path)

Algorithm 3 Process pool driver over a temperature/replicate grid

Validate CONFIG; expand temperature/replicate grid into {JobSpec} with unique seeds and file names
Open summary JSONL; choose worker count; submit each job to a `ProcessPoolExecutor`
In each worker: set thread env vars; import `Ka2d_random_seed`; optionally disable tqdm; run `run_ka2d_mc` with job params
Worker computes mean acceptance, mean energy (and per-particle), final step size; returns record with job metadata and save path
Main process writes each record to summary file and prints concise per-job metrics; tracks completion progress

Algorithm 4 Deterministic KA2D MC with burn-in and interval sampling

Build params θ and config cfg; derive composition counts, box L, shuffled types (fixed type RNG seed), initial positions
Split base seed into NumPy stream (positions) and Numba legacy RNG seed; precompute r^2_{cut}, initial energy E
Burn-in: Run $cfg.burn_in_sweeps$ with autotune to adapt d; do not record sweeps
if $cfg.n_samples = 0$ **then**
  Run $cfg.sweeps$ with requested autotune; return traces and final state
end if
Sampling: Freeze autotune; for each interval of $cfg.sample_interval$ sweeps:
  run sweeps; store wrapped and real positions; compute and store energy per sample
  capture full sweep-by-sweep positions for the first window (optional diagnostics)
Return samples, energies, acceptance trace, counts/types/box, initial/final positions, final d; caller may persist results